
◆ DEMO TASK 2

Inventory Allocation System

(Node.js + React.js + Flutter)

🕒 **Timeline:** Max 3 Days

🎯 **Purpose:** Test problem-solving ability, backend orchestration, and API discipline

📌 WHAT YOU HAVE TO BUILD

1 Database

Create the following tables:

Product

- id
- name
- stock

Order

- id
- productId
- quantity
- status

2 API (EXTREMELY IMPORTANT)

You are allowed **ONLY ONE API**:

`POST /order`

This API must:

- Validate product existence
- Validate available stock
- Deduct stock
- Create order
- Prevent negative stock
- Handle concurrent requests safely

✗ More than one API = **Fail**

3 Backend Structure (MANDATORY)

Backend must follow this structure:

```
routes/  
controllers/  
services/  
repositories/  
models/
```

✗ Missing or mixed separation = **Fail**

4 Frontend (Minimal)

React

- Simple form to place order

Flutter

- Use the same API
- No UI focus

✗ WHAT YOU MUST NOT DO

- ✗ Do NOT create multiple APIs
- ✗ Do NOT split logic into multiple endpoints
- ✗ Do NOT put logic inside controller
- ✗ Do NOT handle stock logic in frontend
- ✗ Do NOT skip edge cases



LOGIC EDGE CASE (MUST HANDLE)

If product stock = 5:

- Order 1 → quantity 3 → success
- Order 2 → quantity 3 → must fail

✗ If this case is not handled correctly = **Fail**



SUBMISSION REQUIREMENTS

- GitHub repository (**commit history mandatory**)
 - README including:
 - Folder structure
 - API flow
 - Concurrency handling explanation
 - Screen recording explaining:
 - Why only one API was used
 - Where business logic is implemented
 - How race conditions are prevented
-



IMPORTANT NOTE (FOR BOTH TASKS)

- This demo task is designed to evaluate **your understanding of logic, architecture, and backend responsibility**.
- You may use documentation or references for syntax, but **direct AI-generated or copy-pasted code is not allowed**.
- During review or interview, you must be able to **clearly explain every part of your code and design decisions**.
- Inability to explain your own implementation will lead to **immediate disqualification**, regardless of output.